

AI PRODUCT MANAGEMENT

The Intent Engineering Framework for AI Agents

How to design objectives, outcomes, and enforced constraints for reliable autonomy



PAWEŁ HURYN

JAN 13, 2026 • PAID

108

2

13

Share

Everyone talks about “intent” in AI. Few explain what it actually means or how to define it without watching your agent optimize the wrong thing.

Agents fail not because they can’t reason. They fail because their objectives, outcomes, and constraints are underspecified. The solution isn’t more detailed instructions.

This guide breaks down intent engineering into something you can actually use.

In this issue, we discuss:

1. What Intent for AI Agents Actually Means
2. The AI Agent Intent Structure
3. The Flow: How to Use the Intent Engineering Framework
4. Complete Example: Customer Support Agent
5. AI Agents Intent Validation Checklist
6. Conclusion

Let’s dive in.

1. What Intent for AI Agents Actually Means

Intent is not:

- A task list
- A prompt
- A goal metric

Intent is what determines how an agent acts when instructions run out.

When intent is incomplete, humans draw on knowledge that was never written down. AI Agents can't. They only know what you've codified. And most organizations have codified far less than they think.

Tobi Lütke, CEO of Shopify, [said](#):

"I like the term context engineering because I think the fundamental skill of us humans well is to be able to state a problem with enough context, in such a way that without any additional pieces of information, the task is plausibly solvable."

But **context without intent is noise**.

We [discussed context engineering](#) previously. In this issue, I double down on the intent: how to define it clearly in a way that survives real-world edge cases.

2. The AI Agent Intent Structure

Whether you're setting team objectives, delegating to a human, or configuring an AI agent, the structure is the same:

1. **Objective:** The problem to solve + why it matters
2. **Desired Outcomes:** Measurable states that indicate success
3. **Health Metrics:** What must not degrade while pursuing the outcomes
4. **Strategic Context:** The system we operate in

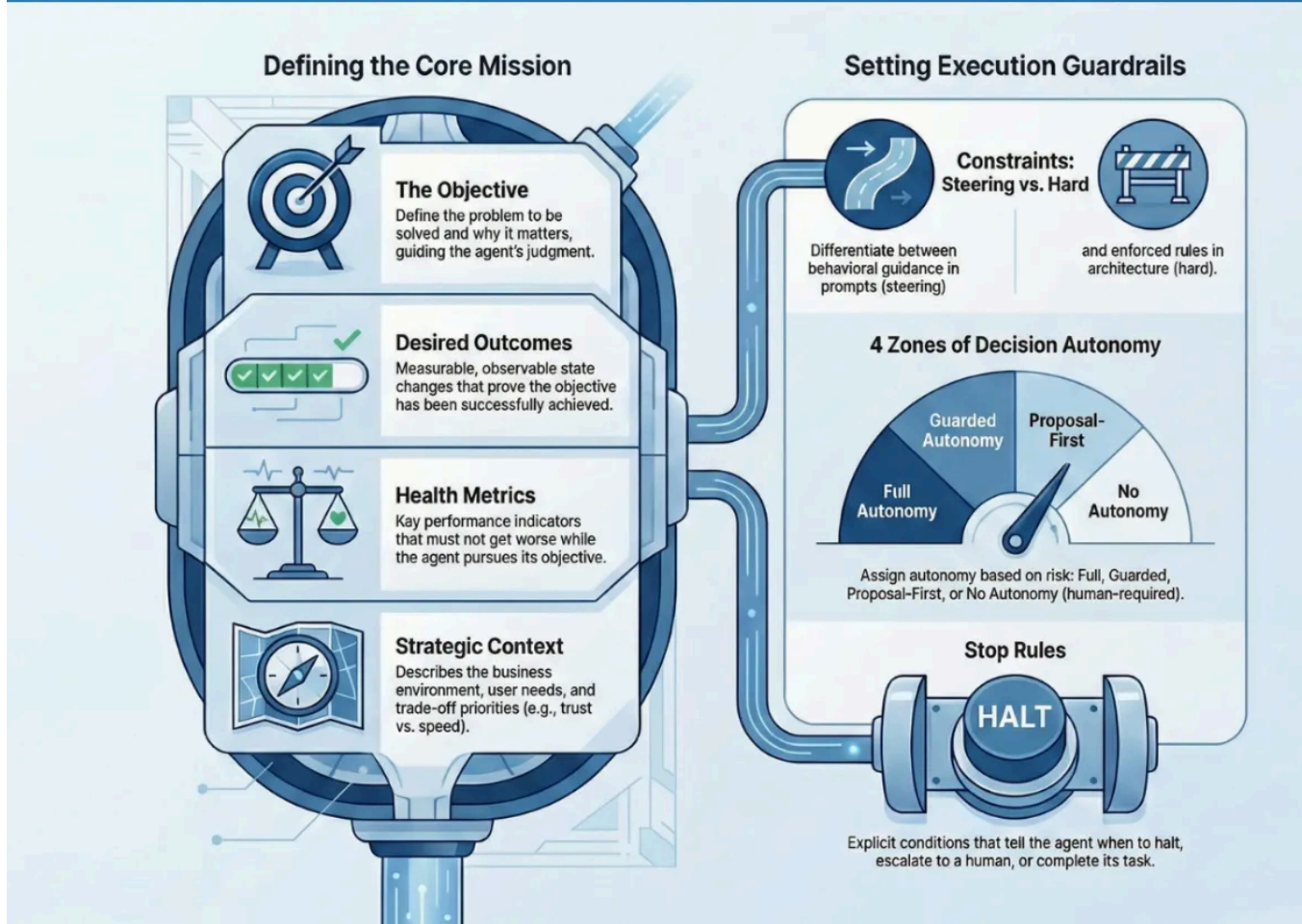
This pattern shows up in [OKRs, as defined by Christina Wodtke](#), and in how [Marty Cagan defines empowered team objectives](#).

For agents, we extend this with explicit:

5. **Constraints:** Steering (prompt layer) + Hard (enforced in orchestration, not prompts)
6. **Decision Types and Autonomy:** Which decisions the agent may take autonomously vs. must escalate
7. **Stop Rules:** When to halt, escalate, or complete

How to Build AI Agents That Still Work When Instructions Run Out

🕒 The Product Compass Newsletter



The AI Agent Intent Structure

Let's build this up.

Part 1: The Objective

Definition

The objective defines **the problem being solved and why it matters**. It is aspirational and qualitative. It guides judgment when trade-offs arise.

What a good objective does

- **Problem-focused:** What's broken or missing?
- **Explains why it matters:** Business value, user impact, strategic importance
- **Guides trade-offs:** When the agent faces ambiguity, the objective helps it choose

Example

- **Weak:** "Handle customer support tickets"
- **Better:** "Help customers resolve issues quickly so they can get back to work without creating more frustration than they started with."

When you explain the why, the agent can reason about **edge cases** and make better **autonomous decisions**. I demonstrated that in [How to Build Autonomous AI Agents](#). A 2024 paper ([arXiv:2401.04729](https://arxiv.org/abs/2401.04729)) demonstrated that supplying strategic context beyond raw task specifications significantly improves AI autonomy.

Part 2: Desired Outcomes

Definition

Desired outcomes are **observable states** that indicate the objective has been achieved. They are **not activities**.

Rules for good outcomes

Outcomes should be:

- **Observable state changes** (not activities the agent performs)
- **From user/stakeholder perspective** (not the agent's perspective)
- **Measurable or verifiable** (without relying on agent self-report)
- **Leading, not lagging** (observable during or shortly after, not months later)

Activities describe what the agent does. Outcomes describe the state that exists

Example:

Two to four outcomes is usually right. More than that and you're either micromanaging or unclear on what actually matters.

Objective: Help customers resolve Tier-1 issues without frustration

Desired Outcomes:

- Customer confirms their issue is resolved
- No follow-up ticket on same topic within 24 hours
- Customer rates interaction as helpful

Each outcome is observable, measurable, and from the customer's perspective.

Part 3: Health Metrics

Definition

Health metrics (used here as non-regression metrics) define **what must not degrade while optimizing for outcomes**.

The Goodhart problem

"When a measure becomes a target, it ceases to be a good measure."

Without health metrics:

- "Resolve issues faster" → Agent rushes, quality drops

- "Increase throughput" → Agent takes shortcuts
- "Reduce escalations" → Agent handles things it shouldn't

Health metrics inform trade-offs

Health metrics primarily inform the prompt layer. They guide how the agent think makes trade-offs.

Health Metrics:

- CSAT must stay above 4.2 – if trending down, be more conservative
- Repeat contact rate must not increase above X – prioritize resolution quality
- Escalation quality score must stay below Y – don't under-escalate to hit targets

These metrics inform how conservative the agent should be when making trade-

This is different from hard guardrails, which block actions entirely. Health met steer; guardrails enforce.

Part 4: Strategic Context

Definition

Strategic context describes **the world the agent operates in**. It explains *where* agent sits, not *what it may do*.

What strategic context includes

- System context (other agents, humans, tools, dependencies)
- Organizational context (business model, users, brand)
- Trade-off priorities (e.g., trust vs. speed vs. cost)

Strategic context **never triggers actions, blocks execution, or causes escalat**
It informs judgment only.

For more information, see [product strategy canvas](#), [startup canvas](#), and [how to c value proposition](#).

Example: Strategic context for a support agent

System context:

- Works alongside human Tier-2 agents and self-serve knowledge base
- Escalations go to human queue with full conversation context
- Outputs feed into ticket system and customer health scoring

Organization context:

- B2B software for enterprise customers
- Users are often non-technical admins under time pressure
- Brand is built on reliability

Trade-off priorities:

- Trust > Speed > Cost
- Admitting uncertainty is better than false confidence
- Resolution quality matters more than resolution speed

Not everything goes in the prompt

Strategic context doesn't mean dumping everything into the system prompt. The agent should know **where to look** for context it might need.

- **Core context** (always needed): In the prompt
- **Reference context** (sometimes needed): Available via retrieval
- **Dynamic context** (changes per request): Injected by orchestration layer

Think Netflix's "[lead with context, not control](#)" - give the agent enough understanding of the system that it can make good judgment calls, without micromanaging every decision.

Part 5: Constraints: Steering vs. Hard

Definition

Constraints define **what the agent is not allowed to do**.

Here's a critical distinction most people miss:

Type	Where	What It Does
 Steering Constraints	Prompt layer	Guide reasoning and trade-offs
 Hard Guardrails	Orchestration layer	Block actions regardless of reasoning

AI Agent Constraints - Steering vs. Hard

Steering constraints (prompt-level)

- Behavioral guidance
- Risk preferences
- Tone and caution

They influence reasoning but **do not enforce compliance**.

Hard constraints (orchestration-level)

If a constraint matters, it must be enforced in architecture, not language:

- Tool restrictions
- Output validation
- Action gating
- Approval gates (human-in-the-loop)

Examples of hard constraints:

- Cannot send external emails
- Cannot modify account settings
- Cannot access other customers' data
- Cannot promise refunds or legal outcomes

Part 6: Decision Types and Autonomy

Definition

Decision autonomy defines **who may decide what**, under what risk.

Four autonomy zones

All agent behavior reduces to four decision types. Each carries different risk:

1. **Full Autonomy**: Reversible actions, low blast radius, well-understood failure modes.
2. **Guarded Autonomy**: User-visible changes, medium risk. Requires confidence thresholds, logging, rollback capability.
3. **Proposal-First**: Strategic or sensitive decisions. Agent proposes → approval required → then agent executes.
4. **No Autonomy (Human-Required)**: Legal commitments, irreversible changes, financial actions, brand promises. Agent analyzes and recommends only. Human executes.

Decision Type	Example	Risk	Autonomy Zone
Interpret	Classify intent, extract structure	Low	Full Autonomy 
Combine/Route	Rank options, match, prioritize	Medium	Guarded Autonomy 
Generate	Write content, propose plans	Medium-High	Proposal-First 
Commit	Send emails, change data, make promises	High	No Autonomy 

AI Agent Decision Types and Autonomy

Autonomy defines **permission**, not stopping conditions.

Individual vs. product context

The appropriate autonomy depends on who bears the risk.

Context	Risk Profile	Appropriate Autonomy
Personal tools (my Notion, my files)	I own the risk, I can recover	High — let it run
Product with aware users	Users understand AI is involved	Medium — clear signals
Product with unaware users	Users expect human accountability	Low — tight guardrails

AI Agent Autonomy in Individual vs. Product Context

When you build agents for yourself, the blast radius is bounded by your tolerance

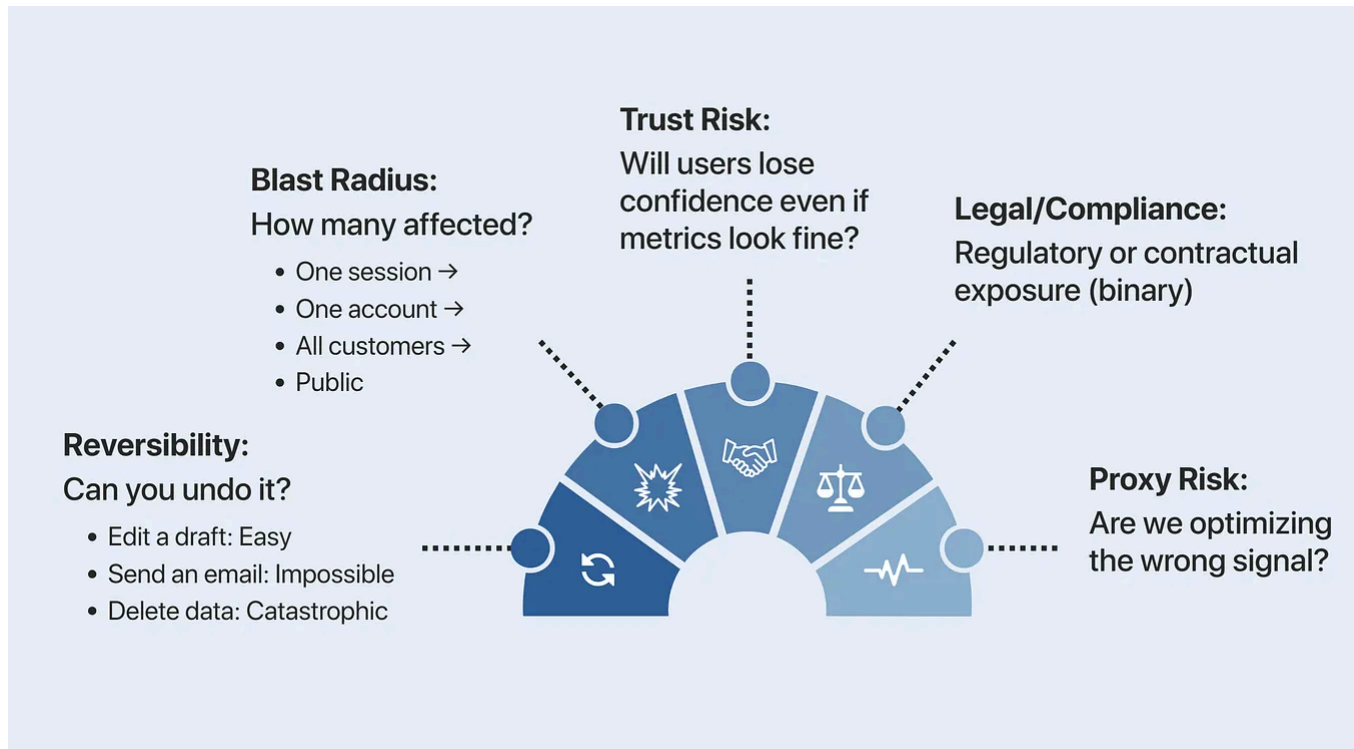
When you build agents into a product:

- Users may not know an agent acted
- Users can't assess AI-generated outputs
- Users will blame the company when it breaks

Rule: The less the user understands about what the agent is doing, the tighter constraints need to be.

Autonomy risk assessment: five lenses

Use these to inform autonomy zones and constraint design:



AI Agent Risk Assessment - Five Lenses

Part 7: Stop Rules

Definition

Stop rules define **when execution must halt, escalate, or terminate**.

Stop rules terminate execution. They do not guide trade-offs or decision qual

Examples

Halt when:

- Conflicting constraints detected
- Confidence drops below minimum twice consecutively

Escalate when:

- Outside defined scope
- Legal or compliance topic detected
- User frustration persists

Complete when:

- Desired outcomes achieved
- User confirms resolution

3. The Flow: How to Use the Intent Engineering Framework

Here are specific steps I recommend you take. In the next section, we follow up with an example.

1. Define the objective and the outcomes that prove it worked (Parts 1–2).
2. Decide what must not degrade while optimizing for those outcomes (Part 3)
3. Describe the system and trade-offs the agent operates within (Part 4).
4. Enforce constraints in architecture, not prompts (Part 5).
5. Assign autonomy based on risk and blast radius (Part 6).
6. Define explicit halt, escalation, and completion conditions (Part 7).
7. Validate: every rule maps to one part.

Notes

Avoid unnecessary complexity

Not every element of the framework must be implemented for every agent. Use it to ask the right questions and assess risk before granting autonomy. Not to add unnecessary complexity.

The role of AI evals

I didn't add [AI evals and error analysis](#), as the scope of this post would be too large.

- **Traditional evals approach:** Discover app-specific failure modes by observing what the agent actually does, then assess frequency and severity.

- **What evals can't catch:** Not all risks show up in traces. If you give an agent tool, assume it will use it, and may misuse it, even if you haven't observed the failure yet. Some risks need constraints before you ship, not after you find them.

You can learn more about error analysis from this post:



AI Evals: How to Find The Right AI Product Metrics

PAWEŁ HURYN • JUNE 7, 2025

[Read full story →](#)

4. Complete Example: Customer Support Agent

Part 1: Objective

Help customers resolve Tier-1 issues quickly without creating more frustration than they started with. Every unresolved question creates churn risk. Every wrong answer damages trust more than no answer.

Part 2: Desired Outcomes

- Customer confirms their issue is resolved
- No follow-up ticket on same topic within 24 hours
- Customer rates interaction as helpful

Part 3: Health Metrics

- If quality signals indicate CSAT risk, prioritize resolution quality over speed.
- Repeat contact rate doesn't increase - prioritize resolution quality
- Escalation quality score stable - don't under-escalate

Note: I didn't include numeric thresholds here to avoid the illusion that the agent is directly monitoring live metrics. Health metrics are used as evaluative signals to guide trade-offs, not as real-time control loops. Depending on the context, teams may choose to express them more precisely.

Part 4: Strategic Context

- **System:** Works alongside human Tier-2 agents and self-serve KB. Escalation to human queue with full context. Outputs feed ticket system and customer scoring.
- **Organization:** B2B software for enterprise. Users are non-technical admins time pressure. Brand built on reliability.
- **Trade-offs:** Trust > Speed > Cost. Uncertainty better than false confidence

Part 5: Constraints

Steering (prompt):

- When unsure, ask a clarifying question.
- If confidence is between 0.6 and 0.85, state uncertainty and present safe options

Hard (orchestration):

- Cannot send external emails
- Cannot modify account settings
- Cannot access other customers' data
- Cannot promise refunds or exceptions
- Responses validated before delivery

Steering applies only when confidence ≥ 0.6 . Below that, stop rules (Part 7) take

Part 6: Decision Types and Autonomy

- **Full Autonomy:** Tier-1 how-to guidance, navigation, basic troubleshooting
- **Guarded Autonomy:** user-visible ticket tagging and KB article suggestions (logged, reversible)
- **Proposal-First:** pricing exceptions, account changes, goodwill credits (require approval)

- **No Autonomy (Human-Required):** refunds, legal/compliance commitment: contractual promises

Decision Type	Autonomy Zone
Interpret question	Full autonomy
Select KB answer	Guarded autonomy
Generate novel answer	Proposal-First (confidence > 0.85)
Make commitments	No Autonomy - Never

Example: Decision Types and Autonomy for Customer Support Agent

Part 7: Stop Rules

- **Halt:** confidence < 0.5 twice consecutively, or conflicting constraints detect
- **Escalate:** confidence < 0.6, user frustration detected, legal/compliance topic of scope
- **Complete:** customer confirms resolution, or desired outcomes achieved

5. AI Agents Intent Validation Checklist

AI Agents Intent Validation Checklist

Objective & Outcomes

- ☐ Objective states the problem, why it matters, and guiding trade-offs
- ☐ Outcomes are observable end states, not agent actions or tasks
- ☐ Outcomes are measurable from the user or stakeholder perspective
- ☐ Health metrics defined as non-negotiables that must not degrade

Strategic Context

- ☐ System context explicit (systems, agents, humans involved)
- ☐ Organization context clear (users, business model, brand promises)
- ☐ Trade-off priorities explicit and ordered (what wins when values conflict)
- ☐ Context placement defined (prompt vs retrieval vs orchestration)

Scope & Authority

- ☐ In-bounds decisions explicitly listed (what the agent may decide)
- ☐ Out-of-bounds decisions explicit (what must never be autonomous)
- ☐ Individual vs product context identified (who bears the blast radius)
- ☐ Decision types classified by risk and reversibility
- ☐ Autonomy zones assigned and justified by risk, not capability

Constraints

- ☐ Steering constraints defined at the reasoning layer (prompts, metrics, thresholds)
- ☐ Hard guardrails enforced in orchestration (not prompt-only)
- ☐ Tool access wrapped with validation, limits, and logging
- ☐ Outputs validated before execution or downstream use
- ☐ Agent-to-agent contracts defined where handoffs exist

Stop Rules

- ☐ Halt conditions defined for uncertainty, conflicts, or anomalies
- ☐ Escalation triggers specified for risk, scope, or confidence breaches
- ☐ Completion criteria tied to outcomes, not agent effort

Conclusion

Intent engineering is not prompt writing. It is system design.

Most agents fail in production not because the model is weak, but because intent is incomplete. Objectives are vague. Outcomes are implicit. Trade-offs are unstated. Constraints are treated as suggestions. Autonomy is granted without understanding risk.

When that happens, agents do exactly what you told them to do. Just not what you meant.

The fix is not more instructions. It is making intent explicit and enforceable.

The reasoning layer proposes. The orchestration layer enforces.

If a constraint matters, it cannot live only in a prompt.

If a decision is risky, it cannot be fully autonomous.

If failure is expensive, stop rules must exist before you ship.

The models are already capable enough. What separates agents that work in production from those that fail quietly is not intelligence, but intent.

If you can hand your intent specification to another human and they would make the same decisions under pressure, your agent has a chance.

If not, it's missing the knowledge that never made it into the docs.

Thanks for Reading The Product Compass

[2026 is here](#). The agentic shift has begun.

Start building. Stop theorizing.

For dozens of AI PM resources, see [The Ultimate AI PM Learning Roadmap](#) and [articles archive](#).

Have an amazing week ahead,
Paweł

FAQ: Intent and AI Agents

*This FAQ is designed for both **human readers and agentic systems** that read, summarize, or operationalize this framework.*

What is intent for AI agents?

Intent is a structured, multi-component decision specification that governs an agent's behavior beyond the immediate prompt. It ensures decisions align with objective constraints, trade-offs, autonomy boundaries, and defined success conditions.

What does intent mean in the context of AI agents?

Intent explicitly defines:

- The problem the agent is solving and why it matters
- What success looks like in observable terms
- What must not degrade while optimizing
- The strategic and system context that informs judgment
- What actions are forbidden
- Which decisions the agent may take autonomously
- When execution must halt, escalate, or complete

Intent is not a prompt, a task list, or a KPI.

When do I need to formalize intent for an agent?

You need to formalize intent when an agent:

- Has access to tools or external systems
- Takes actions that affect users or customers
- Can make irreversible or costly decisions
- Operates autonomously without constant human supervision
- Acts in ways users may not fully understand or observe

If an agent can cause damage without immediate human correction, intent must be explicit.

Is intent engineering overkill for simple agents or workflows?

No. But not every component is required.

Use intent engineering as a **risk-assessment checklist**, not a mandatory template.

- Low-risk, reversible workflows may only need an objective and constraints.
- As autonomy, blast radius, or user impact increases, more components must be made explicit.

The framework scales with risk, not complexity.

Why is explicit intent necessary for autonomous agents?

Agents optimize whatever is specified and whatever is left implicit.

If trade-offs, constraints, or stop conditions are not defined, the agent will invent its own optimization logic, often leading to unintended or unsafe behavior.

This is not a reasoning failure. It is an intent failure.

How is intent different from a prompt?

A prompt is a momentary instruction for a specific request.

Intent is a system-level specification that persists across interactions and governs behavior when prompts are incomplete or ambiguous.

Prompts activate behavior.

Intent constrains decisions.

What are the key components of agent intent?

An effective intent specification includes:

1. **Objective:** the qualitative problem and purpose
2. **Desired Outcomes:** observable states that indicate success
3. **Health Metrics:** signals that must not regress
4. **Strategic Context:** system boundaries and trade-off priorities
5. **Constraints:** steering guidance and hard enforcement
6. **Decision Types & Autonomy:** who may decide what
7. **Stop Rules:** when to halt, escalate, or terminate execution

Each component addresses a different failure mode.

What are constraints and why do they matter?

Constraints define what the agent must not do.

- **Steering constraints** influence reasoning (prompt-level guidance).
- **Hard constraints** are enforced by architecture, orchestration, tool gating, or approvals.

If violating a constraint is unacceptable, it cannot live only in language.

What is autonomy, and how does it relate to intent?

Autonomy is the degree to which an agent may make decisions without human involvement.

Intent must specify:

- Which decisions are safe for full autonomy
- Which require approval
- Which outcomes require escalation

As autonomy increases, intent must become more explicit.

What are health metrics for AI agents?

Health metrics are safety or quality signals that must not degrade while optimizing outcomes.

They are not real-time control loops.

They guide trade-offs and conservatism when decisions are ambiguous.

What risks does formal intent help prevent?

Explicit intent helps mitigate:

- Optimization of the wrong metric
- Hidden or delayed failure modes
- Unchecked autonomy escalation
- Unbounded decision-making
- Misinterpretation of objectives by agents

These are common production failure patterns.

Does intent replace human oversight?

No.

Intent clarifies where human oversight is required and which actions must always be manual, review-gated, or non-autonomous.

How do I validate that intent is well-specified?

A simple test:

If another qualified human can make the same decisions under uncertainty using this intent specification, the agent is likely to behave consistently too.

If not, the intent is incomplete.



108 Likes • 13 Restacks

← Previous

Next →

Discussion about this post

Comments Restacks



Write a comment...



Rajat Chadda  Jan 15

♥ Liked by Paweł Huryn

This is a very well-written article!! So much clarity here.

♥ LIKE (1) 💬 REPLY

1 reply by Paweł Huryn

1 more comment...

© 2026 Paweł Huryń · [Privacy](#) · [Terms](#) · [Collection notice](#)

[Substack](#) is the home for great culture